🧠 1. What is File Handling?

File handling allows a program to **store data permanently** on a disk and retrieve it later.
Python provides built-in functions to create, read, write, and delete files.

📂 2. Types of Files

1. **Text Files** → Store data in human-readable form (.txt, .csv, .py, etc.)
2. **Binary Files** → Store data in machine-readable form (.bin, .dat, .exe, etc.)

🧩 3. File Operations in Python

Python's main function for file handling is:

open(filename, mode)

Common File Modes:

Mode	Description
'r'	Read mode (default) — Error if file not found
'w'	Write mode — Creates new file or overwrites existing one
'a'	Append mode — Adds content to the end of file
'x'	Exclusive creation — Fails if file exists
'b'	Binary mode
't'	Text mode (default)
'r+'	Read and write mode

🧾 4. Basic Example: Writing to a File

```
# writing data to a file

file = open("example.txt", "w")

file.write("Hello, this is my first file handling example!\n")
```

```
file.write("Python makes file handling easy.")

file.close()
```

✅ Creates example.txt and writes text into it.
If the file exists, its content is **overwritten**.

📖 5. Reading from a File

```
# reading data from the file

file = open("example.txt", "r")

content = file.read()

print(content)

file.close()
```

💡 Other Reading Methods:

Method	Description
file.read(size)	Reads entire file or first size characters
file.readline()	Reads one line
file.readlines()	Reads all lines into a list

Example:

```
file = open("example.txt", "r")

print(file.readline())   # First line

print(file.readlines())  # Remaining lines as list

file.close()
```

🪶 6. Appending to a File

```
file = open("example.txt", "a")

file.write("\nThis line is appended later.")
```

```
file.close()
```

Now, new text is **added** instead of overwriting.

🧰 7. Using with Statement (Best Practice)

Instead of manually closing files, Python's with automatically handles it.

```
with open("example.txt", "r") as file:
    data = file.read()
    print(data)
```

✅ Auto closes the file — even if an exception occurs.
This is the **recommended way** to handle files.

📄 8. Reading Line by Line

```
with open("example.txt", "r") as file:
    for line in file:
        print(line.strip())
```

✅ Useful for large files — reads one line at a time.

⚙️ 9. Writing and Reading Binary Files

```
# Writing binary
data = bytes("Python Binary Example", 'utf-8')
with open("binaryfile.bin", "wb") as file:
    file.write(data)

# Reading binary
with open("binaryfile.bin", "rb") as file:
    print(file.read())
```

🧹 10. Checking File Existence and Deletion

```
import os

if os.path.exists("example.txt"):
    os.remove("example.txt")
    print("File deleted!")
else:
    print("File does not exist.")
```

💻 11. Example: Writing & Reading Student Data

```
# Write student names into a file
students = ["Alice", "Bob", "Charlie", "David"]

with open("students.txt", "w") as file:
    for student in students:
        file.write(student + "\n")

# Read and display names
with open("students.txt", "r") as file:
    for line in file:
        print("Student:", line.strip())
```

📜 12. Example: CSV File Handling

```
import csv
```

```
# Write to CSV
with open("data.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["Name", "Age"])
    writer.writerow(["John", 25])
    writer.writerow(["Emma", 30])

# Read from CSV
with open("data.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

✅ 13. Summary

Operation	Function/Mode
Open File	open()
Read	'r', read(), readline(), readlines()
Write	'w', write()
Append	'a', write()
Close File	close() or with block
Binary	'rb', 'wb'
Delete	os.remove()